

今日書くコードが

明日の技術負債になる

Compose開発における設計負債の予防

はじめに

技術負債は「過去の遺産」ではない

- 今日書いたコードが明日の負債になる
- 重要なのは**予防的な設計**
- 今日のテーマ：Compose開発での実践的な予防法

技術負債とは？

Ward Cunningham（1992年）が提唱した概念

「短期的な解決策を選ぶことで生じる、将来の開発コスト」

借金と同じように：

- 元本 = コードの問題
- 利子 = メンテナンスコストの増加

放置すると利息が膨らんでいく

技術負債の2つのカテゴリー

1. 技術選択負債

- ViewからComposeへの移行
- RealmからRoomへの切り替え
- プラットフォーム選択

2. 設計負債

- コードの書き方や構造
- ハードコーディング
- 結合度の問題
- **今日から改善できる！**

今日は**設計負債**に焦点を当てます

問題のあるコード例

```
@Composable
fun UserListScreen(viewModel: UserListViewModel) {
    val users by viewModel.users.collectAsState()
    val isLoading by viewModel.isLoading.collectAsState()
    var showDialog by remember { mutableStateOf(false) }
    var selectedUser by remember { mutableStateOf<User?>(null) }

    LaunchedEffect(Unit) { viewModel.loadUsers() }

    Column {
        TopAppBar(title = { Text("ユーザー一覧") })
        if (isLoading) {
            CircularProgressIndicator()
        } else {
            LazyColumn {
                items(users) { user ->
                    Row(
                        modifier = Modifier.fillMaxWidth()
                            .clickable {
                                selectedUser = user
                                showDialog = true
                            }
                            .padding(16.dp)
                    ) {
                        AsyncImage(/*...*/)
                        // ... 数十行続く
                    }
                }
            }
        }
    }
}
```

設計負債を見つける3つの視点

1. **Preview関数** - 修正箇所の探しやすさ
2. **Side Effect & State** - 状態の見やすさ
3. **責務の分離** - 変更の影響範囲

視点① Preview関数が簡単に書けるか？

先ほどのコードにPreviewを書こうとすると...

```
@Preview
@Composable
fun UserListScreenPreview() {
    // ViewModelが必要...FakeViewModelを作る？
    // 状態の組み合わせが多いと大変
    UserListScreen(viewModel = FakeUserListViewModel())
}
```

問題点

- FakeViewModelを用意する必要がある
- 様々な状態を確認するために複数のFakeが必要
- Previewを書くのが面倒 → ビルド＆実行に頼る結果に

視点② 状態管理が見やすいか？

```
@Composable
fun UserListScreen(viewModel: UserListViewModel) {
    // 画面固有の状態
    var showDialog by remember { mutableStateOf(false) }
    var selectedUser by remember { mutableStateOf<User?>(null) }

    // ViewModelの状態
    val users by viewModel.users.collectAsState()
    val isLoading by viewModel.isLoading.collectAsState()

    // 副作用
    LaunchedEffect(Unit) { viewModel.loadUsers() }

    // UIの定義が数百行...
}
```

問題点：状態管理とUI表示が混在、把握しづらい

視点③ 責務が分離されているか？

1つのComposableに詰め込まれている：

- 状態管理（ViewModel接続、ローカル状態）
- 副作用（データ読み込み）
- UI構築（リスト、ダイアログ）
- イベント処理（クリック、ダイアログ表示）

→ 変更の影響範囲が不明確

→ テストが困難

→ 再利用できない

予防的な設計の実践

原則① Statelessを意識する

```
// Before: Stateful (状態を持つ)
@Composable
fun UserCard(user: User) {
    var expanded by remember { mutableStateOf(false) }
    // ...
}

// After: Stateless (状態を持たない)
@Composable
fun UserCard(
    user: User,
    expanded: Boolean,
    onExpandChange: (Boolean) -> Unit
) {
    // 状態を持たず、パラメータで受け取る
}
```

Statelessのメリット：再利用可能、Preview可、テスト容易

原則② コンポーネントを分離する

```
@Composable
fun UserListScreen(viewModel: UserListViewModel) {
    val users by viewModel.users.collectAsState()
    val isLoading by viewModel.isLoading.collectAsState()
    var selectedUser by remember { mutableStateOf<User?>(null) }

    UserListContent(
        users = users,
        isLoading = isLoading,
        onUserClick = { selectedUser = it }
    )

    selectedUser?.let { user ->
        UserDetailsDialog(
            user = user,
            onDismiss = { selectedUser = null }
        )
    }
}
```

```
@Composable
fun UserListContent(
    users: List<User>,
    isLoading: Boolean,
    onUserClick: (User) -> Unit
) { /* Statelessな実装 */ }
```

原則③ 小さなコンポーネントに分割

```
@Composable
fun UserCard(user: User, onClick: () -> Unit) {
    Row(
        modifier = Modifier.fillMaxWidth()
            .clickable(onClick = onClick)
            .padding(16.dp)
    ) {
        UserAvatar(avatarUrl = user.avatarUrl)
        Spacer(modifier = Modifier.width(12.dp))
        UserInfo(name = user.name, email = user.email)
    }
}

@Composable
fun UserAvatar(avatarUrl: String) {
    AsyncImage(
        model = avatarUrl,
        contentDescription = null,
        modifier = Modifier.size(48.dp).clip(CircleShape)
    )
}

@Composable
fun UserInfo(name: String, email: String) {
    Column {
        Text(name, style = MaterialTheme.typography.bodyLarge)
        Text(email, style = MaterialTheme.typography.bodySmall)
    }
}
```

Previewも簡単に書ける

```
@Preview
@Composable
fun UserCardPreview() {
    UserCard(
        user = User(
            name = "山田太郎",
            email = "yamada@example.com",
            avatarUrl = "https://..."
        ),
        onClick = {}
    )
}
```

```
@Preview
@Composable
fun UserListContentPreview() {
    UserListContent(
        users = listOf(
            User("山田太郎", "yamada@example.com", "..."),
            User("佐藤花子", "sato@example.com", "...")
        ),
        isLoading = false,
        onUserClick = {}
    )
}
```

原則④ 副作用を適切に管理

```
class UserViewModel : ViewModel() {
    private val _uiState = MutableStateFlow(UserUiState())
    val uiState: StateFlow<UserUiState> = _uiState.asStateFlow()

    fun loadUsers() {
        viewModelScope.launch {
            _uiState.value = _uiState.value.copy(isLoading = true)
            val users = userRepository.fetchUsers()
            _uiState.value = UserUiState(
                users = users,
                isLoading = false
            )
        }
    }
}
```

- 副作用（ネットワーク通信）はViewModel/UseCaseで管理
- UIは副作用を持たない純粋な表示器

設計原則がもたらすメリット

1. **修正箇所がすぐ見つかる** : Previewで即座に確認
 2. **変更に強い** : 影響範囲が明確
 3. **テストしやすい** : UIとロジックを独立してテスト
 4. **再利用性が高い** : コンポーネントが汎用的
- **技術負債が溜まりにくい**

責務の分離とAtomic Design

今やったことの本質は**責務の分離**

- 状態管理の責務 → 親コンポーネント / ViewModel
- 表示の責務 → 子コンポーネント (Stateless)

Atomic Designの考え方：

- 小さな単位 (UserAvatar、UserInfo)
- 中くらいの単位 (UserCard)
- 大きな単位 (UserListContent)

階層的に構成することで管理しやすくなる

まとめ

技術負債は「過去の遺産」ではない

今日書くコードが明日の負債になる

設計負債を予防する4つのポイント

1. **Preview**を簡単に書ける設計にする
2. **Stateless**を意識する
3. **責務を分離**する
4. **副作用を適切に管理**する

明日から始められること

- 新しいComposableを書くとき **Stateless**にできないか考える
- **Previewを先に書いてみる**（簡単に書けなければ設計を見直す）
- **100行を超えたら分割を検討**する
- **副作用はUI層から分離**する

小さな意識の積み重ねが、将来の開発効率を大きく変える

ご清聴ありがとうございました！